Chapter 5 · Lesson 2 — Prop Drilling vs Context

Chapter 5 · Lesson 2 — Prop Drilling vs Context

> *Goal: stop being scared of prop drilling. By the end of this lesson you can defend prop drilling in interviews, recognize when Context is genuinely the right tool, and know why most internet advice on this topic is overcooked.*

What is "prop drilling"?

When the orchestrator at the top of the tree has data that a component three levels deep needs, you "drill" the data through every level in between:

```
<SnookerFantasyLeague>
  └── <PlayersTab teams={teams}>          ← receives teams (level 1)
       └── <PlayerDetail teams={teams}>   ← receives teams (level 2)
          └── <MatchPickAnalysis teams={teams}>  ← receives teams (level 3, fin
```

The middle components don't *use* `teams` themselves — they just pass it through. That feels wasteful. It feels like ceremony. Beginners hate it.

It's also fine. Three levels of explicit prop passing is one of the most readable patterns in React.

The internet's bad advice

Search "prop drilling" and you'll find articles screaming *"avoid prop drilling at all costs! Use Context!"* This is wrong for most apps.

The reason it's wrong: **the alternative — Context — has costs the articles never mention**. Specifically:

1. **Context hides data flow.** With drilling you can `Cmd+F "teams"` and see exactly where it goes. With Context, you can't — `useContext(TeamsContext)` could be called from anywhere.
2. **Context's default behavior is to re-render every consumer when the value changes.** Even if a particular consumer doesn't care about the part that changed.
3. **Context requires a Provider somewhere up the tree.** That's a setup tax, plus a "what if the Provider is missing?" test surface.
4. **Context is contagious.** Once you start using it for one piece of data, it's tempting to put everything in Context and end up with a global mess.

For 3-level drilling, **the costs of Context exceed the costs of drilling**. For 8-level drilling, maybe Context wins. Always ask: "what specifically am I trying to avoid?"

When prop drilling is fine (most of the time)

The honest answer: **prop drilling is fine until it's painful**, and "painful" usually means more than 4–5 levels.

Symptoms of "actually too much drilling":

- A component takes a prop and passes it unchanged to literally one child, and that's its only purpose.
- You're passing the same prop through 6+ component levels.
- Adding a new piece of data requires editing 7 files just to thread it through.
- You're naming intermediate props things like `passThroughTeams` because "teams" already means something else at the intermediate level.

Until you hit those symptoms, **drill happily**. It's explicit, debuggable, type-checked, and refactor-friendly.

In our codebase

Maximum drilling depth in this codebase is **3 levels**:

```
<SnookerFantasyLeague>            (level 0)
  └── <PlayersTab teams={...}>    (level 1)
        └── <PlayerDetail>         (level 2)
              └── <MatchPickAnalysis teams={...}>   (level 3, uses)
```

Three levels of `teams`. Each component in between either uses it or passes it. Total cognitive cost: nearly zero. We do not need Context.

Why we DON'T use Context for `teams`

Let's pretend we did. Context-based version:

```
// Set up Context at the orchestrator
const TeamsContext = createContext<TeamWithScores[]>([]);

function SnookerFantasyLeague() {
  const teamsWithScores = useMemo(...);
  return (
    <TeamsContext.Provider value={teamsWithScores}>
      <PlayersTab />
    </TeamsContext.Provider>
  );
}

// Consume in MatchPickAnalysis
function MatchPickAnalysis() {
  const teams = useContext(TeamsContext);
  // ...
}
```

Pros: `<PlayersTab>` and `<PlayerDetail>` don't have a `teams` prop in their signatures.

Cons:

- A new file (`contexts/TeamsContext.tsx`) with a Provider definition and a hook.
- The Provider somewhere in the tree.
- Slightly more cognitive load — readers have to know that `<MatchPickAnalysis>` is not self-contained; it depends on a Provider being present.
- Testing `<MatchPickAnalysis>` in isolation requires wrapping it in a Provider.
- If we accidentally render `<MatchPickAnalysis>` outside the Provider, we get the default value (`[]`) and silent rendering bugs.

For 3 levels and one piece of data, **the drilled version is just better**. Three levels of `teams` props is more readable than the Context version.

When Context IS the right answer

Context excels at **cross-cutting concerns** — data that's needed by many components at unrelated parts of the tree. Three real cases:

1. Theme

```
const ThemeContext = createContext<'light' | 'dark'>('light');
```

Theme is consumed by **every** component (or close to it). Drilling theme through 50 components would be misery. Context is the right call.

2. Authenticated user

```
const UserContext = createContext<User | null>(null);
```

Many components need to know who's logged in: header (show name), nav (show admin links), forms (pre-fill email), etc. The set of consumers is broad and unrelated. **Drilling user through everything is misery; Context wins.**

3. Locale / i18n

```
const LocaleContext = createContext<Locale>('en');
```

Same logic. Every translated string needs the locale; drilling it would be ridiculous.

What these have in common

- Used by **many** components scattered across the tree.
- The data is **stable** (it doesn't change every interaction).
- The consumers are **disparate** — they don't share a parent above the orchestrator.

Compare to `teams` in our app:

- Used by **3** components in a single sub-tree (Players).
- Computed once and stable.
- All consumers are in one feature folder.

`teams` doesn't qualify. `theme`, `user`, `locale` do.

Context's performance footgun

Even when Context is the right architectural choice, it has a **performance** trap.

```
const TeamsContext = createContext({ teams: [], filter: '', setFilter: () =>
```

When you put multiple unrelated values in one Context, **every consumer re-renders when ANY of them changes**. If `setFilter` is called, components that only read `teams` re-render too.

The fix is **splitting Context** into smaller, single-purpose contexts:

```
const TeamsContext = createContext<TeamWithScores[]>([]);
const FilterContext = createContext<{ filter: string; setFilter: (s: string)
```

Or using **separate read and write contexts**:

```
const TeamsContext = createContext<TeamWithScores[]>([]);
const TeamsActionsContext = createContext<{ setSelectedTeam: (...) => void }
```

These are real production patterns. **If you reach for Context, plan for splitting it.**

What about Zustand, Jotai, and friends?

If Context is "too much" but drilling is "too painful," there's a middle layer: **lightweight state libraries**. The two leading options in 2026:

Zustand

```
import { create } from 'zustand';

const useStore = create<{ teams: TeamWithScores[]; setTeams: (t: TeamWithSco
  teams: [],
  setTeams: (teams) => set({ teams }),
}));

// Anywhere in the app:
function MatchPickAnalysis() {
  const teams = useStore(s => s.teams);
```

```
  // ...
}
```

About 1 KB of code. No Provider required. Components subscribe to slices of state. Re-renders are tracked per-slice. **The selector pattern (`s => s.teams`) is the killer feature** — it ensures only components that read changed data re-render.

When to use: more shared state than `useState` can comfortably handle, less than would justify Redux. **Most production apps these days.**

Jotai

```
import { atom, useAtom } from 'jotai';

const teamsAtom = atom<TeamWithScores[]>([]);

function MatchPickAnalysis() {
  const [teams] = useAtom(teamsAtom);
  // ...
}
```

Similar scope, different mental model. State is decomposed into "atoms"; components subscribe to specific atoms. Computed atoms can derive from others. **Smaller, more compositional, slightly more learning curve.**

Redux

The classic. **Boilerplate-heavy.** Modern Redux Toolkit (RTK) makes it bearable, but for new projects in 2026, you reach for it only when you genuinely need:

- Time-travel debugging.
- Action logging / replay.
- Middleware (e.g. observability hooks).
- Strict, auditable state transitions.

For 95% of apps, Zustand or Jotai is enough.

> **Interview answer:** *"For small apps, useState in the parent. Past that, Zustand for most cases — small, no Provider, selector pattern. Context for cross-cutting*

concerns only (theme, auth, locale). Redux only when you need its specific features (time-travel, middleware, action replay).”

The senior decision tree

When considering “where should this state live?”:

```
Is this state used by ONE component?
├── Yes → useState inside that component.
└── No → continue.

Is it used by 2-3 nearby components (sibling-ish)?
├── Yes → Lift to common ancestor; prop drill.
└── No → continue.

Is it cross-cutting (theme, auth, locale)?
├── Yes → Context.
└── No → continue.

Is it medium-shared and changes often?
├── Yes → Zustand or Jotai.
└── No → continue.

Do you need time-travel debugging or middleware?
├── Yes → Redux.
└── No → reconsider — you may have over-thought it.
```

That tree gets you the right answer 95% of the time. Memorize it.

A real-world Context: the dark mode example

Pretend our app had dark mode. The implementation would look like:

```
// contexts/ThemeContext.tsx
'use client';
import { createContext, useContext, useState } from 'react';

type Theme = 'light' | 'dark';
```

```
interface ThemeContextValue {
  theme: Theme;
  toggle: () => void;
}

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = useState<Theme>('light');
  const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');
  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside ThemeProvider');
  return ctx;
}
```

Three things to call out:

The “create + Provider + custom hook” trio

Every Context in production code follows this pattern:

1. `createContext` — defines the shape.
2. A `Provider` component that owns the state.
3. A custom hook (`useTheme`) that calls `useContext` AND throws if no Provider is present.

The custom hook is the small but huge insight. Without it, every consumer writes `useContext(ThemeContext)` directly and silently gets the default value if no Provider wraps them. With the hook, missing Provider throws a clear error.

Default value `null` + assertion in the hook

```
const ThemeContext = createContext<ThemeContextValue | null>(null);

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error(...);
  return ctx;
}
```

The default value is `null`, which is invalid. The custom hook narrows the return type to `ThemeContextValue` (non-null) by throwing if it's null. **This means consumers don't have to handle null themselves** — the hook guarantees a real value or fails loudly.

The Provider pattern

```
<ThemeProvider>
  <App />
</ThemeProvider>
```

The Provider wraps the app (or a sub-tree). Its `value` prop is what consumers receive. **Wherever the Provider is in the tree, that's the boundary** — only descendants of the Provider can use the Context.

Most apps put global Providers in `app/layout.tsx` or right after.

What we DO use that's "globalish"

Even though we don't use Context, there are two pieces of "shared, app-wide" data in our codebase:

Module-level constants

```
// lib/teams.ts
export const TEAMS: Team[] = [...];
```

```ts
// lib/matches.ts
export const ROUND1_MATCHES: Match[] = [...];
```

These are just exported constants. Any file can import them. No React state, no Context. **They're "global" in the sense that they're available everywhere, but they don't change.** That's fine — it's just imported data.

For data that doesn't change at runtime, **module-level constants are the simplest possible "global state."** No hooks, no Providers, no library. Just `import { TEAMS } from '@/lib/teams'` and use it.

Style helper functions

```ts
// lib/constants.ts
export function tabStyle(active: boolean): React.CSSProperties { ... }
```

Same idea — a global helper, but it's a pure function, not state.

The senior takeaway: **a lot of what beginners think needs to be "state" is actually just "imported data" or "imported helpers."** Don't put data in state if it doesn't change. Don't put helpers in Context if they're stateless.

The seven-level drilling test

Here's a thought experiment to know if you've drilled too far:

> Imagine you're inserting a new component between an ancestor and a descendant. Do you have to add a `prop` to your new component just to thread something through that it doesn't use?

If the answer is "yes" and it's the third time you've done it, you're drilling too far. If the answer is "no" or "rarely," drilling is fine.

For our codebase, the answer is rarely. Inserting `<MatchPickAnalysisCard>` between `<PlayerDetail>` and `<MatchPickAnalysis>` would require passing `teams` through it. That's once. Not enough to refactor.

Vibe prompt you would have used

> *"My app's `teams` array is being prop-drilled three levels deep: orchestrator → tab → detail → match-analysis. A teammate suggests using Context to avoid drilling. Should I? Lay out the trade-offs honestly. Default to NOT using Context unless drilling is causing real pain."*

The "default to NOT using Context" line is the trick. Without it the LLM will help you implement Context "for cleanliness" — and you'll regret it.

CHECK YOURSELF

1. **A friend says "you should use Context for `teams` to avoid prop drilling." What questions do you ask before agreeing?**
2. **What's the difference between** prop drilling** and **passing a prop**? When does the latter become the former?**
3. **Why is the "create + Provider + custom hook" trio the standard pattern for Context?** What does the custom hook protect against?
4. **You're building a settings page that needs the user's theme, locale, and authenticated identity. Three Contexts or one?** Justify.
5. **`TEAMS` is a module-level `export const`. It's available everywhere in the app. So is it "global state"?** Why or why not?

When you've answered these, head to **03-building-player-detail.md** — the case study.